```
    "properties": {
      "brand": {
        "type": "text"
      },
      "colour": {
        "type": "text"
      },
      "energy_rating": {
        "type": "text"
      },
      ...
      "user_ratings": {
        "type": "double"
      },
      "price": {
        "type": "double"
      }

    }
  }
}
```

There are no surprises in the `products` definition except that a couple of attributes (`price` and `user_ratings`) are defined as `double` but the rest are declared as `text` fields. Most fields are declared as multiple data types (e.g., `text` and `keyword`) to accommodate working with term-level queries on data (e.g., energy rating or color).

The mapping in listing 11.1 creates an empty `products` index with the relevant schema for the e-commerce electrical products we are about to index. Next, we index a set of sample products.

11.1.2 Indexing products

Now that the schema is ready, let's index our product data set for Elasticsearch, which is available with the book's files. Copy the contents of products.txt and paste them into Kibana. We use the `_bulk` API to index this data. This code snippet shows a sample of the data:

```
PUT _bulk
{"index":{"_index":"products","_id":"1"}}
{"product": "TV", "brand": "Samsung", "model": "UE75TU7020", "size": "75",
"resolution": "4k", "type": "smart tv", "price": 799, "colour": "silver",
"energy_rating": "A+", "overview": "Settle in for an epic..",
"user_ratings": 4.5, "images": ""}
{"index":{"_index":"products","_id":"2"}}
{"product": "TV", "brand": "Samsung", "model": "QE65Q700TA", "size": "65",
"resolution": "8k", "type": "QLED", "price": 1799, "colour": "black",
"energy_rating": "A+", "overview": "This outstanding 65-inch ..",
"user_ratings": 5, "images": ""}
{"index":{"_index":"products","_id":"3"}}
...
```

Now that we've indexed our products data set, let's set the scene by discussing the need for compound queries and why and how they help build advanced queries.